RSI, Feedback and Learning Dynamics

Value, knowledge and learning mechanisms | Literature review

Open the reading map and paper explainers

Recursive self-improvement (RSI) asks whether acquired capabilities can improve subsequent learning. A coding system might modify its tools or the program proposing further improvements. These change different parts of learning. We organize the literature by value, knowledge and learning mechanisms, then examine how feedback supports progress or sustains error.

Category	What changes	Central question
Value and evaluation	Goals, rewards, scoring rules	What makes a change better?
Knowledge and interaction	Tools, experiments, world models	How can evidence correct beliefs?
Learning mechanisms	Updates, architecture, memory, improvers	How can later learning improve?

1. Value and evaluation: rewards are not final goals

Training rewards guide learning; final objectives evaluate its outcome. Singh et al. show how internal rewards can encourage exploration and improve long-term performance beyond immediate-reward optimization. A value function instead estimates return under a given reward. Improving that estimate, rewriting rewards and changing objectives are distinct problems.

With the final objective fixed, reward design has two levels. A trains a policy using reward r; J evaluates that policy. Choose the reward giving the best training outcome:

$$r^* \in \operatorname{argmax}_r \; E[J(A(r))]$$

The expectation averages over training randomness. Eureka repeats this process: a language model writes rewards, reinforcement learning trains policies, and training feedback guides reward revision. Matched-candidate comparisons with one-shot generation, and removal of reward-component statistics, support feedback-guided search. The task objective stays fixed.

Changing the objective changes the standard of improvement

Everitt et al. study utility self-modification. Evaluating changes by the new utility can favor declaring every outcome good. Evaluating predicted behavioral consequences by the current utility gives different incentives. With modification-independent initial utility and beliefs, an initially optimal policy, and their other assumptions, the latter rule preserves optimal behavior for the original objective.

Decision theory also asks whether modification is worthwhile

Computation and modification have costs. Hay et al. ask whether another simulation's expected decision benefit justifies its cost. A Gödel machine changes its program, including its proof searcher, after proving that switching now beats continued search. This guarantee depends on its axioms and provability. Modern RSI systems commonly test changes experimentally instead.

2. Knowledge and interaction: acquiring and using evidence

Choose evidence for the decisions it can improve

Once an objective is set, a learner must decide what to observe or try. Blackwell compares information sources through garbling: one signal can be produced from another by a fixed, state-independent random transformation. With the same objective, no acquisition cost and optimal use of information, retaining the more informative signal cannot lower decision value.

Dual control gives actions two roles: performing the task and learning its unknown dynamics. Control experiments vary exploration costs and the relevance of unknown parameters to test when future learning justifies an immediate sacrifice. Information-directed sampling (IDS) targets which action is best, balancing current expected regret against information about the optimal action. Not every unknown detail deserves equal attention.

The same evidence need not be equally usable

Decision value does not settle whether a bounded learner can extract a pattern. Xu et al. restrict predictors to a family $\mathcal{V}$. Conditional V-entropy is the lowest expected prediction loss available within it:

$$H_{\mathcal{V}}(Y \mid X) = \inf_{f \in \mathcal{V}} E[-\log f(X)(Y)]$$

Here X is the input, Y the outcome, and f(X)(Y) the probability assigned to that outcome. The loss reduction from observing X defines usable information for this family. It is nonnegative when predictors may ignore X. Deterministic preprocessing cannot increase Shannon mutual information with the target, yet it can make a pattern accessible to a limited predictor.

Abbe et al. examine how simple features support learning more complex ones. Terms such as x_1, x_1x_2 and $x_1x_2x_3$ form successive steps. Their merged-staircase condition orders terms so that each introduces at most one new variable; an isolated high-order product lacks these low-order signals. The results concern sparse functions of uniform bits, wide two-layer networks, a mean-field limit and sample sizes of the same order as input dimension. They characterize learning by a specified algorithm.

Learning progress can guide what to experience next

Schmidhuber rewards compression progress: compare how efficiently the old and new compressors encode the same history. The reward reflects a pattern actually learned, rather than novelty alone. The update algorithm can remain fixed while past learning changes the choice of future experience.

Finzi et al. study a different quantity: structure learnable within a computational time limit. A two-part code describes the learned model and the data it leaves unexplained. Among models satisfying the time constraint, choose one minimizing total description length; epiplexity is the model-description part. Board-game pretraining experiments show that such structure helps different downstream tasks differently. Learnable structure, realized learning progress and task improvement therefore require separate evaluation.

Changing tools, skills and world models

Modern systems change their interaction with the world in different ways. Toolformer learns to call existing tools; Voyager turns acquired skills into reusable tools; Memento 3 revises a world model used to predict action outcomes. They address use, reuse and understanding, respectively.

Tools and skills make existing capabilities reusable

Toolformer samples API calls, retaining examples that reduce subsequent text-prediction loss, then trains on them. The authors compare models trained on the same corpus without tool use and disable tools at inference. The extra benefit from calculator access supports the usefulness of the learned calls. What changes is when and how to call; the available APIs and filtering rule remain specified by the researchers.

Voyager writes action code in a game, revises it using environmental feedback and stores successful code in a skill library. Later tasks can call or combine these skills. Removing the library causes exploration to stall earlier; tasks in a new world further test reuse. The growing resource is skill content. Task selection, retrieval and self-verification procedures remain predefined.

World models use past experience to predict outcomes of untried actions

Memento 3 records environmental regularities in a rulebook and translates it into an executable world model. It predicts action outcomes in the model, executes them in the environment and revises rules using feedback. In an ablation without the rulebook, both systems complete the selected games, but retaining it requires fewer environmental actions. Organizing and revising experience can thus improve subsequent interaction.

World models also need correction. Gelman and Shalizi distinguish updating beliefs within a model class from revising the class when it fails to explain observations. This clarifies what replay validation can establish: several models may fit the same history yet disagree about untried actions. Distinguishing them requires further interaction.

3. Learning mechanisms: learning how to update

Tools and world models change what a system can do; meta-learning also changes how it learns. Learned optimizers produce parameter updates from gradient histories. Neural architecture search learns which network designs to propose. Meta-reinforcement learning stores interaction history in a recurrent state to adapt to new tasks. In the last case, test-time weights remain fixed; adaptation occurs in the internal state.

These studies compare learned mechanisms with conventional optimizers, random search or existing learning rules. The outer procedures that train those mechanisms remain designed by researchers. This raises the next question: can the program responsible for producing improvements itself become a target of modification?

Self-modifying learning: improving the improver

STOP and DGM: revised programs participate in later rounds

A better task program and a better program improver require different evaluations. STOP lets an improver rewrite itself, then evaluates the revision by the average performance of downstream programs it improves. Against the initial improver, single-pass inference and greedy iteration, some models benefit while weaker ones can deteriorate. The ability to produce improvements becomes the explicit target.

Darwin Gödel Machine (DGM) uses revised programs both to solve coding tasks and to generate later modifications, retaining an archive of versions. Two controls keep the initial program as the modifier or continue only from the latest accepted version. The former plateaus earlier; the latter is more vulnerable to poor changes. These comparisons support both using acquired capabilities in later modifications and retaining alternative branches for exploration.

HyperAgents also modifies the process that proposes changes

DGM retains a fixed outer procedure for issuing modification instructions. HyperAgents makes both the task program and its improvement program editable, allowing diagnosis, proposal generation and the organization of later changes to evolve. DGM uses coding ability for both task solving and self-modification; HyperAgents also tests transfer to other tasks. The advance concerns what is editable and the evidence for transfer.

To evaluate improvement ability, the authors freeze an improver at test time, let it generate candidates from a specified task program, select on validation data and evaluate on test data. Transferred improvers yield larger gains on new tasks. However, task code also transfers, and initial failures on mathematics include output-format errors. The evidence supports system-level transfer without fully isolating the improver’s contribution.

Empirical acceptance differs from a proof of benefit

A Gödel machine requires a proof before changing itself. DGM and related systems try changes and evaluate task outcomes; even temporarily weaker versions may be retained for further revision. Foundation models, external evaluation and some selection rules remain fixed. These conditions make recursive improvement experimentally testable and define the scope of the evidence.

Continual learning requires retaining the ability to learn

Dohare et al. ask whether a network can still learn new tasks after prolonged training. This capacity, called plasticity, can decline; resetting low-utility units mitigates the problem. This differs from forgetting old knowledge. The reset rule is designed by researchers, but the result identifies a requirement for RSI: continuing feedback is useful only if the system can still learn from it.

4. How feedback supports or limits self-improvement

Berk–Nash links chosen actions to supporting evidence

Can feedback correct current judgments? Berk–Nash equilibrium allows misspecified subjective models. Actions determine observed responses; beliefs support models fitting these responses best; actions maximize expected payoff under those beliefs. Best fit minimizes prediction error weighted by action frequencies:

$$\Theta^*(\rho) = \operatorname{argmin}_\theta \; E_{a\sim\rho} \; KL\big(Q_W(\cdot|a) \;\|\; Q_\theta(\cdot|a)\big)$$

Here ρ gives action frequencies, θ indexes models, and the two Q distributions describe actual and subjective feedback. KL measures predictive discrepancy. Beliefs support only Θ* and weight the models' predicted payoffs accordingly. Every action taken must maximize this expected payoff. Fitting and optimal action jointly sustain the equilibrium.

Self-confirming equilibrium highlights errors about untried actions; Berk–Nash also allows incorrect explanations of observed feedback. Exploration and model revision therefore address different problems. Equilibrium need not imply convergence. In their Bayesian setting with a fixed model class, Fudenberg, Lanzani and Strack show that if actions converge, their limit must be optimal under every belief supported on the best-fitting models.

GANs and model collapse: discrimination and data sources

With real samples available, limited-discriminator theory asks what differences an evaluator can detect. Arora et al. show that generators lacking substantial diversity can still fool restricted discriminators. Reward overoptimization shows how optimizing a proxy score can widen its divergence from a reference score. Feedback requires an evaluator able to distinguish relevant errors.

Model collapse asks who produces the next generation of data. When the previous model becomes the source, sampling omissions and fitting errors can pass to descendants. Data accumulation changes this transmission by retaining original and historical data. Together with selective real feedback in Berk–Nash and real-sample comparisons in GANs, these studies distinguish feedback sources, coverage and discrimination as influences on correction.

Why board games and code can use self-generated experience

Our feedback guide for board games and code develops two examples. AlphaZero chooses its own games, but rules determine outcomes; its network guides search, and search and game outcomes train the network. CodeRL generates programs, but compilation and tests produce execution results, which a critic turns into training signals. Producing candidates is different from deciding that they are correct.

In board games, search turns consequences of known rules into training targets. In code, execution turns proposed programs into concrete results. Clear, repeatable checks let both systems reuse self-generated experience to improve later choices.

Deployment can also change the feedback distribution

Performative prediction studies data changes caused by deployment. With strong convexity, joint smoothness and sufficiently weak distributional response, repeated retraining can stabilize without minimizing post-deployment risk. Miller et al. instead learn how data respond to deployment and use that response in optimization. Feedback can change with the model's actions as well as evaluate them.

Guide to the paper explainers

Topic guide: How feedback can correct a learner (including board games and code)

Blue names open our English explainers. Original papers and reading locations appear at the end of each explainer. The Berk–Nash guide explains both belief fitting and optimal action.

Why Internal Rewards Can Differ from the Final Objective

Eureka: Improving Reward Programs through Training Feedback

Which Objective Should Evaluate a Change of Objective?

Metareasoning: When Is More Computation Worthwhile?

The Gödel Machine's Criterion for Self-Modification

Blackwell on how information improves decisions

Dual control and the learning value of actions

Information-directed sampling to find the best action

Why the same evidence is more usable for some learners

How simple features create a path to further learning

Choosing experience through compression progress

Epiplexity and structure accessible under limited computation

How Toolformer learns to use existing tools

How Voyager turns accumulated skills into resources for later learning

How Memento 3 uses a world model to improve the next interaction

Why posterior updating cannot replace model criticism

Learned Optimizers: Learning How to Update Another Model

Neural Architecture Search: Learning to Propose Better Models

Meta-Reinforcement Learning: Adaptation with Fixed Weights

How STOP evaluates a program that produces improvements

How DGM lets descendant programs continue self-modification

How HyperAgents makes the improvement process editable

Why continual learning also requires plasticity

Self-confirming equilibrium: why accurate feedback can leave mistaken beliefs intact

Berk–Nash equilibrium: how feedback can sustain a mistaken view

Endogenous misspecified learning: which self-consistent states can persist?

Performative prediction: how a model changes its next training data

Outside the Echo Chamber: optimizing performance after deployment feedback

Model collapse: why learning only from the previous generation loses content

Data accumulation: why retaining history changes self-training error

Reward overoptimization: why proxy scores rise while reference scores fall

Finite discriminators: indistinguishable distributions need not be identical